

Linux Programming Assignment 6

1. Which command is used to list the contents of a directory? Justify with proper example. (CO1)

The `ls` command is the standard utility used to list the contents of a directory in Linux. It displays filenames, directories, and, with various options, detailed metadata such as permissions, ownership, size, and modification timestamps. For example, running `ls -l /home/student` will produce a long listing format that shows each item's type and permissions, number of links, owner, group, file size in bytes, last modification date, and filename. This detailed format is especially useful for administrators who need to audit directory contents, verify file permissions, or check file sizes and timestamps without invoking multiple commands.

2. Write the command to create a new directory named `123test_dir`. (CO1)

To create a new directory called `123test_dir`, use the command `mkdir 123test_dir`. The `mkdir` utility creates the specified directory with default permissions determined by the current user's `umask`. If the parent directories do not exist and you need to create them in one step, you can include the `-p` flag—for example, `mkdir -p /home/student/123test_dir`—which creates any missing parent directories automatically. However, for a directory in the current working directory, `mkdir 123test_dir` suffices to immediately create it.

3. What is the purpose of the `sed` command? Justify with proper example. (CO1)

The `sed` (stream editor) command is designed to perform basic text transformations on an input stream, whether from a file or piped data. It allows pattern-based search and replacement, line insertion or deletion, and text filtering without opening an interactive editor. For instance, to replace all occurrences of "foo" with "bar" in a file named `data.txt`, you would run `sed 's/foo/bar/g' data.txt`. This command reads each line from `data.txt`, applies the substitution expression `s/foo/bar/g` (where `s` stands for substitute, `g` for global replacement within the line), and outputs the modified text to standard output. By combining `-i`, you can perform in-place edits: `sed -i 's/foo/bar/g' data.txt` directly modifies the file.

4. Which distinct command is used to display one-line descriptions of any commands? (CO1)

The `whatis` command is used to display concise, one-line descriptions of commands, drawing from the manual page database. Running `whatis ls` will output something like “ls (1) - list directory contents,” giving you a quick reminder of each command’s purpose. Internally, `whatis` operates by searching the manual page database for entries that match the specified name exactly, making it ideal for situations where you know a command’s name but need a brief refresher on its function without paging through a full manual.

5. Write the command to create an empty file named “notes.txt”. (CO1)

To create an empty file called `notes.txt`, you can use the `touch` command: `touch notes.txt`. If `notes.txt` does not exist, `touch` creates it with zero length; if it already exists, `touch` updates its access and modification timestamps without altering its contents. This command is the simplest way to ensure a file’s existence for scripting or placeholder purposes.

6. Differentiate between `grep` and `awk` commands with an example. (CO2)

Both `grep` and `awk` are powerful text-processing utilities, but they serve different primary purposes. `grep` specializes in searching for lines matching a specified pattern, offering simple filtering capabilities. For example, `grep "error" logfile.txt` prints all lines containing the string “error.” It is fast and ideal for quick pattern searches. `awk`, on the other hand, is a full-fledged programming language for text processing, enabling complex field extraction, arithmetic operations, and conditional logic. For instance, `awk -F ":" '{ if ($3 > 1000) print $1 }' /etc/passwd` uses colon as a delimiter and prints usernames (first field) whose UID (third field) exceeds 1000. This level of data parsing and conditional handling goes well beyond `grep`’s scope, making `awk` the tool of choice for structured text transformations and reports.

7. Write the command to give read, write, and execute permission to the owner of a file script.sh. (CO1)

To grant the file owner full permissions—read, write, and execute—on `script.sh`, use `chmod u=rwx script.sh` or the equivalent numeric form `chmod 700 script.sh`. The symbolic notation `u=rwx` explicitly sets the user’s permissions, leaving group and others

unchanged, while 700 in octal notation translates to `rwX-----`, ensuring that only the owner can read, modify, and execute the file.

8. How is `chown` different from `chgrp`? Give one example for each. (CO1)

`chown` changes the ownership of a file or directory by setting both the user and optionally the group, whereas `chgrp` changes only the group ownership. For example, `chown alice:developers project.txt` changes both the file's owner to "alice" and its group to "developers." In contrast, `chgrp developers project.txt` changes only the group to "developers," leaving the owner unchanged.

9. A user complains that they cannot execute a file even though it exists in their directory.

How would you troubleshoot this using `ls -l`, `chmod`, and `whoami`? (CO3)
First, run `ls -l filename` to inspect the file's permission string; if the execute bit for the user (indicated by `x` in the owner's triplet) is missing, that explains why the file cannot be run. For example, a listing like `-rw-r--r--` shows no execute permission. Next, use `whoami` to confirm the current user matches the file's owner; if the user is different, they may lack necessary group or other execute permissions. Finally, grant execute permission to the owner with `chmod u+x filename` or, if appropriate for all users, `chmod +x filename`. Re-running `ls -l` should now show an `x` in the permission string, allowing execution.

10. Design a command pipeline to: find all `.log` files modified in the last 2 days in `/var/log`, display them on screen, and save the results into a file `recent_logs.txt` using `tee` command. (CO4)

A suitable pipeline combines `find` with `tee` to both display and record results:

```
find /var/log -type f -name "*.log" -mtime -2 -print | tee recent_logs.txt
```

Here, `-mtime -2` selects files modified within the last two days, `-print` outputs their paths, and `tee recent_logs.txt` displays the list on screen while writing the same output to `recent_logs.txt`. This single command efficiently satisfies both visibility and persistent logging requirements.